

AUTOMOTIVE DIAGNOSTIC AND ACTIVITY MONITORING

Travis Samson Fernandes

Department of Computer Engineering
Padre Conceicao College Of Engineering(Goa University)
Verna,Goa
21ce72.travis@pccgoa.edu.in

Chetan Naik

Department of Computer Engineering
Padre Conceicao College Of Engineering(Goa University)
Verna,Goa
21ce17.chetan@pccgoa.edu.in

Mangesh Phadte

Department of Computer Engineering
Padre Conceicao College Of Engineering(Goa University)
Verna,Goa
21ce36.mangesh@pccgoa.edu.in

Gaurang Madkaiker

Department of Computer Engineering
Padre Conceicao College Of Engineering(Goa University)
Verna,Goa
21ce20.gaurang@pccgoa.edu.in

Abstract—The Automotive Diagnostic and Activity Monitoring (ADAM) system is a comprehensive platform engineered to deliver real-time diagnostics and behavioral insights for four-wheeled vehicles. Designed as both a dynamic monitoring tool and a forensic black box, ADAM integrates multiple sensing modules—including GPS, MPU6050 (gyroscope and accelerometer), and a vibration sensor (SW-420)—to capture vehicular dynamics with high fidelity. Central to the system is its interface with the vehicle's Electronic Control Unit (ECU) via the On-Board Diagnostics (OBD-II) port, enabling access to critical engine parameters and diagnostic trouble codes. Sensor data is persistently recorded onto local storage (SD card) and synchronized with a PostgreSQL database for further processing.

Leveraging this multi-source dataset, ADAM employs a suite of machine learning models for four primary functions: (1) engine health assessment, (2) fuel consumption analysis, (3) driver behavior profiling, and (4) predictive maintenance. Results are rendered through a user-centric mobile application featuring dynamic graphs and charts, offering intuitive visualization of diagnostic metrics. Additionally, an integrated AI-powered chatbot enhances system interactivity by assisting users in interpreting results and suggesting corrective actions. The ADAM system exemplifies a scalable and intelligent automotive analytics solution, promoting vehicle safety, operational efficiency, and data-driven decision-making for both drivers and fleet operators.

Index Terms—Raspberry pi 5, On Board Diagnostic(OBD), Global Positioning System(GPS), ECU(Electronic Control Unit)

I. INTRODUCTION

In recent years, the demand for intelligent vehicle monitoring systems has grown significantly, driven by the need to enhance driver and passenger safety, improve fuel efficiency, reduce emissions, and enable real-time diagnostics. With the global emphasis on climate change and the increasing integration of automation in the automotive sector, monitoring systems have evolved from simple sensor loggers to intelligent platforms capable of advanced data processing and analysis.

One of the key aspects of such monitoring is the ability to retrieve and interpret diagnostic and performance-related data from vehicles. This is crucial not only for preventive

maintenance but also for post-accident forensic analysis and insurance claims. Recognizing these needs, we developed the **Automotive Diagnostic and Activity Monitoring (ADAM)** system, which utilizes the vehicle's On-Board Diagnostics (OBD-II) interface in conjunction with multiple sensing modules to create a holistic monitoring ecosystem.

The **OBD-II standard** was introduced in 1996 by the Society of Automotive Engineers (SAE) and mandated by the U.S. Environmental Protection Agency (EPA) to regulate vehicle emissions. It consists of a 16-pin standardized connector found in all modern light-duty vehicles, serving as a critical access point for monitoring emission-related data and overall vehicle health. This interface provides standardized access to sensor data, engine diagnostics, and fault codes from the vehicle's Electronic Control Unit (ECU)—the central controller that governs all operational aspects of a vehicle.

A. OBD-II and Parameter Identifiers (PIDs)

The OBD-II standard supports several communication protocols, including:

- ISO 15765-4 (CAN-BUS) – widely adopted since 2008
- ISO 14230-4 (KWP2000)
- ISO 9141-2
- SAE J1850 VPW
- SAE J1850 PWM

This study focuses on the ISO 15765-4 (Controller Area Network – CAN) protocol, which is now the default protocol in most vehicles due to its robustness and speed. OBD-II communication occurs via hexadecimal-coded requests sent to the ECU, which responds with encoded data or Diagnostic Trouble Codes (DTCs). These codes are categorized into:

- **DTCs (Diagnostic Trouble Codes)** – Identify faults and malfunctions in vehicle subsystems.
- **PIDs (Parameter Identifiers)** – Used to request live sensor data and real-time vehicle parameters.

The following table lists commonly used PIDs in OBD-II diagnostics, along with their descriptions and typical byte responses:

TABLE I
COMMON OBD-II PARAMETER IDENTIFIERS (PIDs)

PID	Parameter	Description / Units
0C	Engine RPM	Revolutions per minute (2 bytes)
0D	Vehicle Speed	Speed in km/h (1 byte)
05	Engine Coolant Temp	Temperature in °C (1 byte)
0F	Intake Air Temp	Temperature in °C (1 byte)
11	Throttle Position	Percentage (1 byte)
10	MAF Air Flow Rate	Grams/sec (2 bytes)
2F	Fuel Level Input	Percentage (1 byte)
42	Control Module Voltage	Voltage in volts (2 bytes)

By utilizing these PIDs, ADAM is capable of extracting a rich set of telemetry from the vehicle, which is further processed using embedded logic and machine learning algorithms for deeper insights into engine health, fuel efficiency, driver behavior, and predictive maintenance.

II. LITERATURE REVIEW

In recent years, there has been a surge in research focused on the design and development of On-Board Diagnostic (OBD) systems, alongside the application of machine learning (ML) algorithms for predicting engine performance, optimizing fuel consumption, and detecting system anomalies. This section reviews key works in these domains and highlights existing research gaps relevant to the development of the ADAM system.

A. Design and Development of On-Board Diagnostic Circuits

The work presented in [1] outlines the implementation of an OBD system that collects real-time vehicular data such as engine speed, coolant temperature, and diagnostic trouble codes (DTCs) using a KEAZ128 microcontroller. The system emphasizes efficient data collection from the ECU and lays the groundwork for embedded diagnostic units.

In another study [2], a multi-sensor data logger is developed, which includes GPS, an Inertial Measurement Unit (IMU), camera input, and OBD data acquisition. Data is logged locally every 100 ms using a BeagleBoard-XM platform powered by the TI OMAP3530 processor. This architecture allows for synchronized multi-modal vehicle monitoring, enabling rich data capture for analysis and event reconstruction.

The system proposed in [3] uses the ELM327 integrated circuit for interfacing with the OBD port, coupled with GPS data acquisition. Data is transmitted over Wi-Fi using the Carambola2 module and stored on a remote SQL server, highlighting the importance of cloud integration and remote diagnostics for modern vehicles.

B. Review of Machine Learning Algorithms

A comprehensive survey presented in [4] reviews a range of ML models used in vehicle diagnostics and engine control. It explores algorithms such as Artificial Neural Networks (ANN), Support Vector Machines (SVM), Random Forests

(RF), and Adaptive Neuro-Fuzzy Inference Systems (ANFIS). The paper also notes the emerging relevance of Reinforcement Learning (RL) and Deep Reinforcement Learning (DRL) in optimizing fuel economy, controlling emissions, and enabling real-time adaptive control.

In the domain of network diagnostics, [5] introduces an ML-based fault diagnosis framework for Self-Organizing Networks (SONs). While this study is focused on mobile networks, it employs techniques such as Softmax-based Neural Networks and SVMs for multiclass classification, which can be analogously applied to vehicular diagnostic systems. Feature extraction is achieved using Key Performance Indicators (KPIs) and Performance Management Counters (PMCs), and the proposed solution is robust against both single and compound system faults.

Another study [6] investigates the use of feedforward multilayer perceptron ANNs to detect and classify internal combustion engine faults. The study compares various training algorithms—including backpropagation, Levenberg–Marquardt, quasi-Newton, and the newly proposed Smooth Variable Structure Filter (SVSF). Using crank-angle-converted vibration data from a V8 engine, the system successfully detected issues such as worn bearings and abnormal piston noise. Notably, the SVSF algorithm achieved an accuracy of 97% and was able to detect and classify faults within 30 minutes, offering a cost-effective and fast solution for diagnostics.

In [7], the authors present OBD-SecureAlert, a CAN-bus-based anomaly detection system designed to address the lack of security in in-vehicle networks. The system employs a Hidden Markov Model (HMM) to detect deviations in vehicle behavior by analyzing real-time parameters such as speed and engine RPM. Using a sliding window approach, it classifies sequences as either normal or anomalous based on predefined thresholds. Attack scenarios were simulated by injecting malicious data into the CAN bus, demonstrating the system's ability to detect cybersecurity threats effectively.

Summary: While significant progress has been made in both OBD system design and ML-based diagnostics, there remains a gap in integrating low-cost hardware, real-time edge analytics, secure telemetry, and intuitive user interfaces in a single cohesive system. The ADAM system addresses these limitations by combining embedded diagnostics, sensor fusion, machine learning, and mobile-based visualization in a lightweight and accessible architecture.

III. SYSTEM ARCHITECTURE AND METHODOLOGY

The proposed Automotive Diagnostic and Activity Monitoring (ADAM) system is a modular and scalable solution that integrates hardware sensors, software analytics, and cloud-based infrastructure to collect, process, and visualize real-time vehicular data. The architecture is designed to support both diagnostic analysis and driver behavior profiling, offering both forensic insight and real-time feedback through a mobile dashboard.

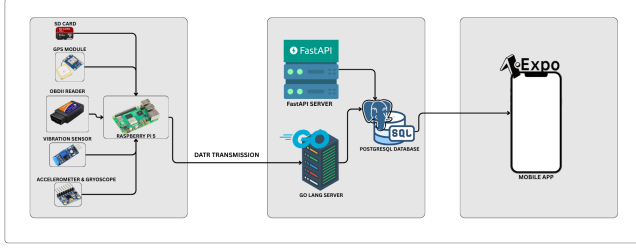


Fig. 1. System architecture of the proposed Automotive Diagnostic and Activity Monitoring (ADAM) system.

A. System Overview

The ADAM system consists of four key components:

- **Hardware Layer:** In-vehicle data acquisition using OBD-II and external sensors.
- **Edge Device and Local Storage:** Data logging using Raspberry Pi and SD card.
- **Cloud-Based Machine Learning Pipeline:** Preprocessing, inference, and analytics.
- **Mobile Application:** User-facing interface for visualization and alerts.

Figure 1 illustrates the overall flow of information from sensor input to final user display via the mobile app.

B. Hardware Integration

The system is built around a Raspberry Pi 5, which interfaces with a standard OBD-II adapter supporting CAN (ISO 15765-4) and K-Line (ISO 9141-2) protocols. In addition to retrieving internal engine diagnostics, the system integrates external sensors such as:

- **Neo-6M GPS module** – For geolocation and speed tracking.
- **MPU-6050 IMU** – To measure acceleration and orientation.
- **SW-420 vibration sensor** – To detect abnormal vibrations indicative of mechanical faults.

All raw sensor data is logged to a local microSD card, enabling forensic data retention in case of network unavailability or vehicle incidents.

C. Machine Learning-Based Data Processing

To enhance vehicle monitoring capabilities, four supervised machine learning models were developed, focusing on:

- 1) **Driver Behavior Analysis**
- 2) **Fuel Consumption Estimation**
- 3) **Predictive Maintenance**
- 4) **Engine Health Performance**

These models are trained on structured vehicular datasets and are periodically updated through retraining pipelines.

D. Cross-Platform Mobile Application

A mobile application, developed using the Expo framework, acts as the primary interface for end-users. It presents real-time diagnostic summaries, trends over time, and user-specific notifications using dynamic visualizations. The app is compatible with both Android and iOS, offering accessibility to a wide user base.

E. Model Training and Feature Importance Analysis

To classify engine health based on sensor features, a Random Forest classifier was trained using a domain-labeled dataset named `Sensor-count_Breeze-Verna-Version4.xlsx`. After preprocessing, including trimming column headers and dropping incomplete entries, the dataset was split into features and labels.

An 80:20 train-validation split was applied, followed by training a 100-tree Random Forest model. Performance was evaluated using standard metrics such as precision, recall, and F1-score. Feature importance scores were then extracted and visualized, enabling interpretation of the most impactful sensor inputs. The trained model was serialized as `engine_model.joblib` for deployment on the Raspberry Pi.

IV. OBD-II AND DIAGNOSTIC PARAMETERS

On-Board Diagnostics II (OBD-II) is a standardized system implemented in all vehicles manufactured after 1996, primarily aimed at monitoring vehicle emissions, engine performance, and subsystem faults. Developed by the Society of Automotive Engineers (SAE), the OBD-II specification includes a 16-pin connector that grants access to the vehicle's Electronic Control Unit (ECU).

A. OBD-II Functionality in ADAM

In the ADAM system, the OBD-II interface serves as a primary data acquisition point, capturing key engine parameters and fault indicators in real time. The Raspberry Pi communicates with the OBD-II adapter via USB or UART, retrieving information encoded in hexadecimal format.

Two major types of codes are used for ECU communication:

- **Parameter IDs (PIDs):** Used to request real-time sensor data (e.g., speed, RPM, fuel level).
- **Diagnostic Trouble Codes (DTCs):** Used to indicate errors or faults in vehicle subsystems.

B. Parameter ID (PID) Communication

When a PID request is sent to the ECU (e.g., '010C' for RPM), the ECU responds with a data frame in hexadecimal. This frame includes the requested values, which are decoded using standard formulas. For example:

- PID '0C' (Engine RPM): Response = '41 0C 1A F8' → $RPM = ((1A \times 256) + F8) / 4$
- PID '0D' (Vehicle Speed): Response = '41 0D 3C' → $Speed = 60 \text{ km/h}$

C. Diagnostic Trouble Codes (DTCs)

DTCs provide encoded fault messages that indicate malfunctions in systems such as ignition, emissions, or engine timing. A typical DTC (e.g., 'P0302') can be decoded as:

- P – Powertrain
- 03 – Misfire
- 02 – Cylinder 2

In ADAM, DTCs are parsed, logged, and flagged in the mobile app to prompt maintenance actions.

D. Supported PID Table

Table II provides an overview of commonly used PIDs retrieved by the ADAM system.

TABLE II
COMMON OBD-II PARAMETER IDENTIFIERS (PIDs)

PID	Parameter	Units / Description
0C	Engine RPM	Revolutions per minute
0D	Vehicle Speed	Kilometers per hour
05	Coolant Temperature	Degrees Celsius
0F	Intake Air Temp	Degrees Celsius
11	Throttle Position	Percentage
10	MAF Air Flow Rate	Grams per second
2F	Fuel Level	Percentage
42	Control Module Voltage	Volts

Through OBD-II, the ADAM system acquires a rich dataset that serves as the foundation for analytics, predictive diagnostics, and personalized feedback to users.

V. MACHINE LEARNING WORKFLOW FOR ENGINE CONDITION ANALYSIS

Algorithm 1 Training and Evaluation of Classification Model

- 1: **Input:** Expert dataset (Sensor-of-Breeza-v4.xlsx)
- 2: **Output:** Trained model and feature ranking
- 3: Load and clean dataset (strip headers, remove NaNs)
- 4: Split data into features (X) and labels (y)
- 5: Perform 80:20 train-validation split
- 6: Train Random Forest with 100 trees
- 7: Evaluate model with classification metrics
- 8: Rank features by importance and plot top 10
- 9: Save model as `engine_model.joblib`

Algorithm 2 Prediction and Engine Condition Summary

- 1: **Input:** `engine_model.joblib`, `test data Breeza_bits.csv`
- 2: **Output:** Predicted labels and summary
- 3: Load model and test data
- 4: Clean and align test data with model features
- 5: Predict engine condition labels
- 6: Count occurrences of each label
- 7: Determine overall status: GOOD, BAD, or Balanced
- 8: Plot label distribution

VI. RESULTS

A. Internal Evaluation on Expert Dataset

The trained Random Forest classifier was internally evaluated using an expert-labeled dataset. The dataset was partitioned into training and validation subsets using an 80:20 split. Table III presents the classification performance metrics obtained on the validation set.

TABLE III
CLASSIFICATION REPORT ON VALIDATION DATA

Class	Precision	Recall	F1-Score	Support
bb	0.85	0.85	0.85	48
gg	0.95	0.95	0.95	137
Accuracy	0.92 (185 samples)			
Macro Avg	0.90	0.90	0.90	185
Weighted Avg	0.92	0.92	0.92	185

The results indicate an overall accuracy of **92%**, with the `gg` class achieving slightly higher precision and recall compared to the `bb` class.

To identify the most influential sensor parameters contributing to the classification decisions, feature importance scores were extracted from the trained Random Forest model. Table IV lists the top two most significant features.

TABLE IV
TOP INFLUENTIAL FEATURES BASED ON IMPORTANCE SCORES

Feature	Importance Score
<code>RUN_TIME</code>	0.7149
<code>Timestamp_OBD</code>	0.2851

A visual representation of feature importance is depicted in Fig. 2. The plot highlights that `RUN_TIME` is the most dominant feature influencing the classification output.

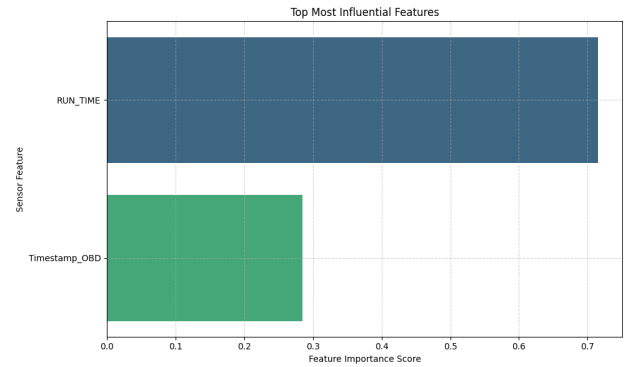


Fig. 2. Top Sensor Features Influencing Model Predictions

During visualization, a future deprecation warning from the Seaborn library was noted regarding the use of the `palette` parameter without an accompanying `hue` assignment. This issue will be addressed in subsequent revisions to maintain compatibility with future library updates.

Summary: The classifier demonstrated high predictive performance on expert data, with `RUN_TIME` emerging as the

most critical sensor parameter influencing engine condition classification.

B. Prediction Results and Distribution Analysis

After deploying the trained engine condition classification model on real-world sensor data obtained from the `Breeza_bits.csv` dataset, the following results were observed.

The model expects two input features for prediction:

- `Timestamp_OBD`
- `RUN_TIME`

The test dataset was cleaned to remove missing values and the required features were extracted accordingly. Predictions were then performed using the loaded model.

The distribution of predicted class labels is presented in Table V. It indicates the frequency of each predicted condition label within the test dataset.

TABLE V
PREDICTION COUNT PER CLASS

Label	Count
gg (Good)	675
bb (Bad)	249

A visual representation of the prediction distribution is shown in Fig. 3. This bar chart highlights the number of occurrences for each predicted class label.

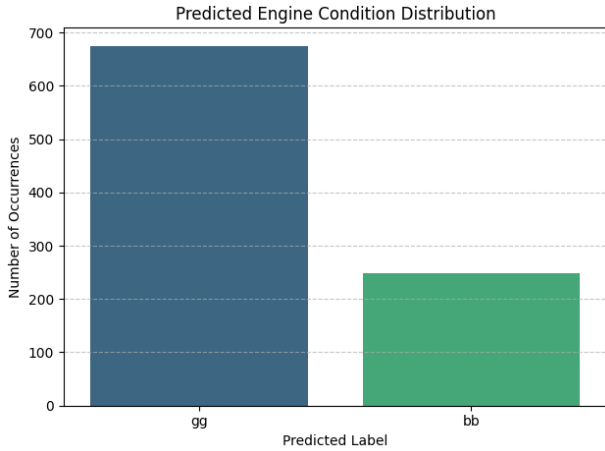


Fig. 3. Predicted Engine Condition Distribution

During visualization, a future deprecation warning was noted in the Seaborn library indicating that passing the `palette` parameter without a corresponding hue assignment will be deprecated in version 0.14.0. This will be addressed in future implementation updates to ensure compatibility.

Based on the prediction results:

- The majority of instances (675) were classified as `gg` (Good condition)
- 249 instances were classified as `bb` (Bad condition)

Hence, it can be concluded that the overall condition status of the test dataset is **Good**, as the number of healthy predictions outweighs the fault detections.

VII. CONCLUSION

The Automotive Diagnostic and Activity Monitoring (ADAM) System presents comprehensive, scalable, and intelligent diagnostics solutions. It integrates multiple hardware modules, machine learning models and a user-friendly mobile interface to offer real-time vehicle diagnostics, driver behavior analysis, and predictive maintenance. By combining OBD-II with additional sensor inputs and processing them using the Raspberry pi 5 it collects and stores data on the cloud and provides insights through a mobile app with multiple functionalities, including an AI chatbot. It not only aids in proactive maintenance, improved fuel efficiency, and personalized user experience but also contributes to safer and greener driving.

REFERENCES

- [1] P. R. Sawant and Y. B. Mane, "Design and Development of On-Board Diagnostic (OBD) Device for Cars," in *Proc. 4th Int. Conf. on Computing Communication Control and Automation (ICCUBEA)*, Pune, India, 2018, pp. 1–4, doi: 10.1109/ICCUBEA.2018.8697833.
- [2] D. L. Nguyen, M. E. Lee, and A. Lensky, "The design and implementation of new Vehicle Black Box using the OBD information," in *Proc. 7th Int. Conf. on Computing and Convergence Technology (ICCCCT)*, 2012, pp. 1281–1284.
- [3] R. Malekian, N. R. Moloisane, L. Nair, B. T. Maharaj, and U. A. K. Chude-Onkonkwo, "Design and Implementation of a Wireless OBD II Fleet Management System," *IEEE Sensors Journal*, vol. 17, no. 4, pp. 1154–1164, Feb. 15, 2017, doi: 10.1109/JSEN.2016.2631542.
- [4] L. F. I. Havugimana, B. Liu, F. Liu, J. Zhang, B. Li, and P. Wan, "Review of Artificial Intelligent Algorithms for Engine Performance, Control, and Diagnosis," *Energies*, vol. 16, no. 3, art. no. 1206, 2023, doi: 10.3390/en16031206.
- [5] J. B. Porch, C. Heng Foh, H. Farooq, and A. Imran, "Machine Learning Approach for Automatic Fault Detection and Diagnosis in Cellular Networks," in *Proc. 2020 IEEE Int. Black Sea Conf. Commun. Netw. (BlackSeaCom)*, Odessa, Ukraine, 2020, pp. 1–5, doi: 10.1109/BlackSeaCom48709.2020.9234962.
- [6] R. Ahmed, M. El Sayed, S. A. Gadsden, J. Tjong, and S. Habibi, "Automotive Internal-Combustion-Engine Fault Detection and Classification Using Artificial Neural Network Techniques," *IEEE Trans. Veh. Technol.*, vol. 64, no. 1, pp. 21–33, Jan. 2015, doi: 10.1109/TVT.2014.2317736.
- [7] S. N. Narayanan, S. Mittal, and A. Joshi, "OBD SecureAlert: An Anomaly Detection System for Vehicles," in *Proc. 2016 IEEE Int. Conf. Smart Comput. (SMARTCOMP)*, St. Louis, MO, USA, 2016, pp. 1–6, doi: 10.1109/SMARTCOMP.2016.7501710.